

Chapter 1 — Writing Notes

Project Framework, State of the Art, and Needs Analysis

Points to make and facts to use. Write the sentences yourself.

How to use this: read a section's notes, close the page, write. Drafting with notes in front of you produces reshaped notes, not your own writing. Write badly first and fix afterwards — the blank page costs more than the editing. One idea per paragraph. State the point, then the evidence, not the reverse.

Target for the whole chapter: 12–15 pages.

Introduction to the chapter

[~½ page, 2 paragraphs]

Job: tell the reader what this chapter does and why it matters

Para 1

- where the project came from, what it was asked to fix
- names: the host institution, the client, the document
- flag that most of the chapter is about how the document gets used today

Para 2

- the requirements at the end weren't decided up front
- they're what's left after watching the current process fail
- → that's why failures come first, requirements after
- closes with: existing tools, why none fit, what was proposed

The point to land: this chapter isn't a survey — it's where the specification comes from

1.1 General Framework of the Project

[~½ page + table]

Job: the administrative framing, quickly

- final-year project, Master's in Data Sciences, Polytech Intl
- six-month internship; ~4 months on the build
- rest: framing the subject, learning the source material, writing this

Then 1–2 sentences: what kept you on it. Three angles, pick one:

- (a) two problems pulling opposite ways — extraction rewards rules you can check against a page; language models give flexibility with no guarantee of faithfulness
- (b) a page a human reads easily breaks every library you point at it — finding out why meant learning to read the file the way the printer wrote it
- (c) the interesting problem wasn't building a chatbot — it was working out what a chatbot must *not* be allowed to do when the subject is who controls money

□ **Table 1.1 — Framework of the project.** Rows: project type, host institution, programme, academic year, duration, client organisation, academic supervisor, professional supervisors, source document, scope processed.

1.2 Host Organisation

[~1½ pages]

1.2.1 Presentation of Polytech Intl

Job: establish the institution; keep it factual, not promotional

- École Polytechnique Internationale Privée de Tunis
- founded 2013 (verify on the school's own site)
- Rue du Lac d'Annecy, Les Berges du Lac 1, Tunis
- founded largely by institutional figures from **banking, insurance, industry**

- programmes: computer science (big data, business intelligence, software engineering, IT-finance, embedded systems), mechatronics, industrial engineering, financial engineering, telecommunications, architecture
- international mobility — partner schools in Europe, Asia, North America
- member of the Agence Universitaire de la Francophonie (verify)
- teaching built around projects + compulsory final internship

Then connect to your project:

- Data Sciences Master's spans stats/ML, data engineering, and the software side of making an analysis actually run
- final project expected to show all three
- → that's why this went to a deployed application, not a clean dataset

1.2.2 Project Context and the Client Organisation

Job: who the client is, and why an institution like that needs a DAM at all

Positioning — get this exactly right:

- Polytech Intl **hosted and supervised**
- AfDB was the **client**: supplied the source document and the requirements
- do *not* describe yourself as working at or within AfDB anywhere

AfDB facts:

- founded 1964, HQ Abidjan, Côte d'Ivoire
- 81 member countries — 54 African, 27 non-regional
- purpose: economic development and social progress across regional member countries
- Group = the Bank + African Development Fund (concessional lending) + Nigeria Trust Fund
- RDGN: Tunisia, Algeria, Morocco, Mauritania, Libya — hub in Tunis (confirm official wording with Yasser or Surya)

□ **One sentence only you can write:** how the subject reached you — who proposed it, how they described the problem.

Optional nice line: a school founded partly by people from banking, supervising a project built for a bank.

Closing paragraph — why a real client changed the work:

- material couldn't be tidied to suit the parser
- correctness = what the printed pages say, not a convenient metric
- every extraction claim in the report is checkable against its page
- → that constraint runs through Chapter 2

□ **Table 1.2 — The client at a glance.** Rows: institution, founded, HQ, membership, purpose, Group entities, regional office concerned, regional hub.

1.3 Study of the Existing System and Critical Analysis

[~5 pages]

1.3.1.1 Definition and stakes of the DAM

[2–3 paragraphs]

- delegation framework answers: for a given institutional act, who may do which part
- expressed as a matrix
- rows = activities, hierarchical: chapters → processes → tasks → child tasks → threshold variants (keyed by money band)
- columns = roles: country office staff → sector managers → directors → VPs → Board
- cell = code saying what that role does for that activity

□ **Table — authority notation.** I / C1,C2 / C3,C4 / R / A / (i)

Two traps in the notation — both matter later:

- “C” covers two unrelated ideas split only by the digit: C1,C2 = check and verify; C3,C4 = consult (Board / independent function)
- the informed marker isn’t a letter at all — it’s parenthesised, and behaves differently at every processing stage

1.3.1.2 Governance and compliance requirements

[1–2 paragraphs]

- the DAM isn’t advisory — it’s the reference internal audit checks against
- exists partly because public and multilateral funding brings governance expectations

Three consequences for any system built on it:

1. a wrong answer surfaces at audit, not during the work
2. completeness = correctness: naming the approver but omitting a mandatory verifier is incomplete in a way that misleads, though nothing in it is false
3. traceability: an answer you can’t verify gives you no basis for relying on it

The point to land: consequence 2 is where your FR4 comes from — flag it

1.3.1.3 Challenges in consulting it in practice

[2–3 paragraphs]

- tables are wide → role headers rotated 90° to fit the page
- reader must map a sideways label to a code centimetres below it
- and get the adjacent column right too — neighbouring roles often differ on the same activity
- superscript footnote digits sit against the codes; telling them apart requires knowing which is which
- some rows hold no responsibilities at all — just a cross-reference, so you restart in a different chapter
- new staff need the document most and are least equipped to navigate it

The point to land: consequence: people ask a colleague instead, or copy what was done last time. Neither is auditable. Both propagate errors quietly.

1.3.2 Study of Existing Systems

Opening: four categories exist, none works, each fails for its own reason, and those reasons become the spec.

1.3.2.1 Manual consultation and document search

[2 paragraphs]

- today: open PDF, Ctrl+F
- full-text search = character string matching
- works if you know the exact title or the identifier; close to useless otherwise
- “approve” → hundreds of hits, page order, no relevance ranking
- half-remembered name → nothing unless wording matches exactly
- **deeper point:** search finds *places*; you wanted an *answer*
- it says page 24; it can’t say who approves, because that needs: page holds a table + headers rotated + letter in cell = a role’s authority

1.3.2.2 Generic document QA assistants

[2–3 paragraphs]

- named: ChatPDF, NotebookLM, Microsoft Copilot, file-upload chat modes

- mechanism (state it accurately — you'll be asked): split into passages → embed as vectors → retrieve nearest to question → model writes from those. This is RAG; use the term.
- works on prose; fails here
- failure is **structural, not model quality** — don't let a reader think a better model fixes it
- flatten page → spatial relationships destroyed
- rotated header becomes a loose string attached to nothing
- retrieved passage can hold the activity name AND the letter A, having lost whose column the A was in
- answer is gone before the model sees anything

□ **Worth 15 minutes:** upload 2–3 DAM pages to ChatPDF or NotebookLM, ask “who approves 2.126”, screenshot it. Wrong or vague → becomes a figure, and turns this subsection from argument into evidence.

1.3.2.3 Commercial document-extraction services

[2 paragraphs]

- named: Azure Document Intelligence, AWS Textract, Google Document AI, LlamaParse
- be fair: markedly better at tables than a general library; the serious alternative to hand-written rules

Three reasons ruled out:

1. per-page cost + sending an internal governance document to a third party — an institutional decision, not a student's
2. tuned for invoices, receipts, contracts, forms — not a bespoke authority matrix with rotated headers and superscript-qualified codes
3. **no account of why a cell came out wrong.** A rule from a measured character offset is checkable against its page. A confidence score isn't.

The point to land: reason 3 is your strongest and connects to the thesis's whole argument — give it the most room

1.3.2.4 Critical analysis

[2–3 paragraphs + table]

- none fail from poor engineering — each fails on an assumption that doesn't hold
- search assumes → user can phrase it in the document's vocabulary
- assistants assume → flattening preserves meaning (false for positional tables)
- extraction assumes → document resembles their training forms
- GRC platforms (4th category) assume → institution re-enters its rules into their data model = change programme, not a tool

The shared failure — put weight here:

- none has any concept of being wrong
- search returns results, model returns text; neither separates “answer I can support” from “answer I can't”
- for a compliance reference that distinction is the whole point

□ **Comparison table.** Columns: Search · Assistants · Extraction · GRC · This work. Rows: preserves positional meaning; answers vs locates; traceable to a source row; refuses when it can't answer; surfaces mandatory obligations; tolerates typos; document stays in-house; no per-page cost; deployable here.

Don't mark a competitor “No” on something you haven't tested — use Partly or N/A and you're safe if asked.

The point to land: read down your own column — that list *is* the specification. Say so, then point at 1.5.

1.4 Functional Needs Analysis

[~3 pages, mostly tables]

1.4.1 Functional requirements

Short intro para: these weren't invented — each corresponds to something a user currently does by hand, or a failure observed in 1.3.

□ **Table.** Core (FR1–FR10):

- FR1 direct identifier lookup — full confidence, no ambiguity
- FR2 natural-language activity lookup + reports its confidence
- FR3 authority-role queries (approve / review / check / initiate / inform)
- FR4 mandatory obligation surfacing — check/verify + informed parties, whatever was asked
- FR5 cross-reference resolution — follow the pointer automatically
- FR6 typo and phrasing tolerance
- FR7 honest refusal — unknown id, out of scope
- FR8 conversational continuity — follow-ups without restating the subject
- FR9 structural analytics — counts, code distribution, top roles, per-chapter
- FR10 multilingual questions — FR/ES/PT/AR, without translating role names or ids

Second phase (FR11–FR16): authenticated access; voice input; document attachment with a distinct trust boundary; fully localised interface incl. RTL; conversation export as PDF with provenance; read-only sharing by QR code.

1.4.2 Non-functional requirements

□ **Table.**

- NFR1 factual fidelity — no role/code/qualification not in the source. **Takes precedence over everything else.** Say that explicitly.
- NFR2 traceability — every answer attributable to a specific record
- NFR3 availability without external services — degrade to templates, don't fail
- NFR4 response time — seconds on free hosting → rules out re-parsing per request
- NFR5 reproducibility — same facts every run, independent of model sampling
- NFR6 portability — identical on laptop and host
- NFR7 usable on a phone
- NFR8 credential protection — no recoverable passwords, tokens expire
- NFR9 upload safety — type and size limits, never executed

1.4.3 Project constraints

□ **Table.** Constraint → consequence:

- no labelled training data → ruled out supervised model; pushed to lexical retrieval
- document scope: 250+ pages, 79 processed → rest as future work
- no infrastructure budget → free tier → forced build-time caching
- no ground truth → correctness established against real pages → set the pace of the whole data understanding phase

1.5 Proposed Solution

[~2 pages]

1.5.1 General presentation

- the core idea: separate *what the facts are* from *how they're said*
- facts ← structured knowledge base, parsed once offline, rules derived from the document's own geometry, validated against printed pages
- phrasing ← language model, given facts already retrieved, restricted to rewording
- between them: a verification step
- any answer that dropped or altered a role name is rejected → deterministic template shown instead
- user still gets a correct answer, just less conversational

The point to land: this is the mechanism that makes a generative component usable at all here

1.5.2 Global system architecture

Four layers:

1. offline extraction — PDF → validated node set, cached, shipped with the app
2. knowledge — node set as a directed graph + lookup and search over it
3. agent — interprets questions, resolves to activities, builds deterministic answers, applies the model + verification
4. presentation — REST interface + authenticated web client (chat, dashboard, attachment, export, sharing)

□ **Figure: fig1_architecture.png**

Worth a sentence: extraction happens offline and is cached, so the running app never opens the PDF. It loads a prepared node set at startup. That was a deployment decision — Chapter 5 covers what happened when it wasn't.

1.5.3 Selected technologies

□ **Table.** Technology → role. Python; pdfplumber (character-level coordinates — required, not convenient); Pydantic (schema validation at construction); networkx (directed multigraph — a role can hold several responsibilities on one activity); scikit-learn (TF-IDF + cosine); FastAPI; React + Vite; Groq + Ollama (behind one interface, so either or neither); Web Speech API; python-jose + passlib; ReportLab; statsmodels; Docker; Render.

For each, give the reason not just the name. “pdfplumber, because the superscript rule needs per-character vertical offset and font size” beats “pdfplumber for PDF extraction”.

Conclusion to the chapter

[~½ page, 1–2 paragraphs]

- the DAM answers an important question; the way it's published makes the answer expensive
- manual consultation is slow and error-prone because the meaning lives in the layout
- generic assistants fail for the same reason — with the added risk that they fail *fluently*
- the requirements above respond to both
- the architecture separates retrieval from phrasing so a model's flexibility can be used without inheriting its unreliability
- hand off: next chapter examines the source document and what makes extracting it hard

Your numbers, for cross-reference: 250+ page document, 79-page subset — 327 records (3 chapters, 35 processes, 138 tasks, 102 child tasks, 49 threshold variants) — 1,776 responsibility entries — 131 roles — 0.56% unresolved — 5.48 responsibilities per record — 29.63% with none — 14 cross-references — graph 459 nodes / 2,108 edges — 374 tests across 23 files.